

# type: A Lightweight Type System for R Functions and Objects

by *Ethan Sansom*

**Abstract** R is a dynamically typed language in which the absence of static type checking, combined with flexible data coercion rules and limited built-in type introspection functionality, allows mistyped input data to silently produce unexpected behaviour or raise difficult to debug errors that obscure their source. The `type` package addresses this by providing a lightweight, composable type system for R. Types in `type` are standalone user-defined objects that explicitly define the predicates an object must satisfy to be considered a valid member of that type, and may be used to annotate function arguments, return values, and variables. When a type-annotated object fails to satisfy its type, an error is raised immediately, identifying the mistyped value and the constraint it violated. Types and typed functions are self-documenting, printing a human-readable description of their requirements without additional authoring effort, and type checking can be added or removed incrementally without restructuring existing code. This paper motivates the design of `type` against R's existing type facilities, describes the package, and compares `type` to existing tools for runtime type checking in R.

## 1 Introduction

R is dynamically typed, meaning that variables can be bound to any data structure and type errors are discovered at runtime rather than before a program executes. This flexibility is a useful feature, allowing for rapid prototyping and an interactive, exploratory style of programming which is well suited to statistical programming and data analysis (Turcotte et al., 2020). Functions built into R further support this flexibility by attempting to coerce input data into a valid type, sometimes with a warning, potentially producing unexpected results.

```
# A list coerced to NA
mean(list(1, 2, 3))
#> [1] NA

# A zero length number coerced to -Inf
max(numeric())
#> [1] -Inf

# Integers coerced to characters
c(1L, 2L, "A")
#> [1] "1" "2" "A"
```

Functions that do not coerce their inputs frequently fail to validate the expected types of their input arguments immediately, instead proceeding until an internal operation fails, often several calls later. These types of failures are difficult to debug, as they produce an opaque error message that references an internal function call rather than the original source of the error.

```
# Coercing a function to a factor
as.factor(mean)
#> Error in `unique.default()`:
#> ! unique() applies only to vectors

# Taking the time difference of TRUE and TRUE
difftime(TRUE, TRUE)
```

```
#> Error in `as.POSIXct.default()` :  
#> ! do not know how to convert 'time1' to class "POSIXct"
```

In contrast to the flexible approach to input coercion used by R's built-in libraries, the R development community has adopted a convention of defensive programming, wherein the expected type of function arguments is checked explicitly at the function boundary. This pattern is illustrated by the **targets** package (Landau, 2021) for building analysis pipelines:

```
targets::tar_crew <- function(store = targets::tar_config_get("store")) {  
  # Boundary input type validation  
  tar_assert_allow_meta("tar_crew", store)  
  tar_assert_scalar(store)  
  tar_assert_chr(store)  
  tar_assert_nzchar(store)  
  # Implementation...  
}
```

Functions using this pattern are much easier to debug, as they report type errors immediately and in the correct context. `sum()` for example reports invalid input types with a clear error message:

```
sum("A")  
#> Error in `sum()` :  
#> ! invalid 'type' (character) of argument
```

Across packages, defensive programming is practiced inconsistently, with some using custom assertions, as in **targets**, others using dedicated assertion libraries such as **checkmate** (Lang, 2017), and some using hand-written if guards or `stopifnot()`. While these approaches catch and report type errors early, they produce errors and error messages of inconsistent quality and provide no shared vocabulary of which conditions constitute a valid object of a given type. Further, expected argument types must either be inferred from inspecting the body of a function itself, or be specified in function parameter documentation, which is often incomplete (Vidoni, 2022).

The **type** package addresses these limitations through a small, composable type system, where a type is defined as a set of predicates, called traits, that a member object must satisfy. Types are themselves objects, allowing them to be readily modified, composed, and inspected by users. Any object or function argument may be restricted to a given type using a lightweight annotation syntax. Violations of an object's or argument's type restrictions raise an error that clearly identifies the type constraint that was violated, the value that violated it, and the locations at which the violation occurred. When printed, a type displays a human readable description of its requirements. Likewise, typed functions display the restrictions which must be satisfied by their typed arguments, making types and type expectations self-documenting without additional effort from users.

The remainder of this paper is organized as follows. Section 2 provides a background on R's existing object types and why R's built-in type validation is often insufficient for detecting malformed objects. Section 3 describes the features of the **type** package and provides examples of its usage. Section 4 covers related work, Section 5 discusses limitations of the package, and Section 6 concludes the paper.

## 2 Object Types in R

Every R object has one of 25 base types, which correspond to the C-level storage representation of the object. The base types include the atomic vector types ("logical", "integer", "double", "character", "complex", "raw"), recursive types ("list", "pairlist"), and function and environment types, as well as others, such as the "promise" type, which are rarely

encountered by users. The base type of an object is accessible via `typeof()`, which returns the base type of its input as a string. The base type of an object is immutable and always accurately describes the object's storage.

Any R object may additionally carry attributes, arbitrary named metadata stored as a named list and accessible via `attr()` or `attributes()`. The `class` attribute has a special purpose with regards to an object's type, as it determines the dynamic dispatch behaviour of generic functions in R. The class of an object `x` is accessed via `class(x)`, which returns a character vector of one or more class names attached to `x`. A generic function, such as `print()` or `summary()`, delegates its implementation to a class-specific method, rather than operating on an object directly. When a generic function `foo` is called with an input `x`, R's method dispatch system searches for an implementation `foo.class_name` for each class name returned by `class(x)`, falling back to the default method `foo.default` if no implementation is found<sup>1</sup>.

Together, `typeof()` and `class()` are the primary means of collecting runtime object type information in R, with `typeof()` describing the structural properties of an object and `class()` determining the contexts in which an object may be used. In practice, however, typing in R most closely resembles a nominal type system, in which an object's type is determined by class name rather than by structure. `inherits(x, "data.frame")` returns `TRUE` if and only if `"data.frame"` appears in `class(x)`, regardless of the data `x` contains. This is the method used in R's built-in type testing predicate, `is.data.frame()`, for data frame objects:

```
base::is.data.frame <- function(x) {  
  inherits(x, "data.frame")  
}
```

Unlike languages with a static type system, such as Java or TypeScript, where an object's class name implies a structural obligation, the class of an object in R is mutable and unvalidated, allowing structurally malformed data to pass nominal type membership tests and later be used in invalid contexts. In some cases, this will result in unexpected behaviour without error:

```
x <- function() {}          # Declare `x` as a function  
class(x) <- "data.frame"    # Modify the class of `x`  
is.data.frame(x)           # Data frame test passes  
#> [1] TRUE  
  
# Uses the print.data.frame method  
print(x)  
#> NULL  
#> <0 rows> (or 0-length row.names)
```

In other cases, the malformed object can result in an opaque error that fails to identify the root cause of the failure:

```
class(x) <- "Date"  
  
# Uses the print.Date method  
print(x)  
#> Error in `as.POSIXlt()`:  
#> ! cannot coerce type 'closure' to vector of type 'double'
```

In practice, the structural requirements of a given class in R are typically maintained by the constructor function for that class. For example, the `data.frame()` constructor always constructs a structurally valid data frame object with the following properties:

---

<sup>1</sup>R's S4 generics additionally support multiple dispatch, where method selection is determined by the classes of more than one argument.

- A class attribute of “data.frame”.
- A base type of “list”.
- Every element within the base list is a vector of the same length.
- An attribute “row.names” with a base type of either “character” or “integer”, of equal length to elements in the base list, and containing no missing values.

Note that the validity of the data frame type depends on both nominal (`class()`) and structural (`typeof()`) constraints, as well as value-level constraints, e.g. a lack of missing values in the “row.names” attribute. The reliance on constructors to create valid objects is limiting in two ways. First, objects are only guaranteed to be valid at the time of construction and may be manipulated later without error. Second, the requirements of a structurally valid object are implicit in the object’s constructor, and cannot be readily inspected, reused later in an object’s lifetime, or used within a function for defensive programming.

### 3 The Type Package

#### 3.1 Types and Traits

The primary component of the type system in **type** is the trait predicate, an [S7](#) (Vaughan et al., 2025) object with three methods:

- `trait_test()` returns TRUE if an object satisfies the trait and FALSE otherwise.
- `trait_describe()` returns a character vector describing the characteristics of an object which satisfies the trait, used when printing a type or typed function to display its requirements to the user.
- `trait_diagnose()` returns a character vector describing the characteristics of an object which failed to satisfy the trait, used in error messages for objects with an invalid type.

A type is an [S7](#) object containing a list of traits. An object is said to be a member of a type if and only if it passes every trait in the list. All types are built on the `t_any` type, which contains no traits and is satisfied by all objects.

Three families of traits cover the main forms of runtime object type introspection in R:

- Nominal traits check class membership. The trait `classed("factor")` is satisfied if an object inherits from the “factor” class.
- Structural traits check the underlying storage type of an object. `bare_typed("list")` is satisfied if the `typeof()` of an object is “list” and the object has no class attribute, distinguishing between bare lists and classed objects built on top of a list, e.g. data frames.
- Value-level traits inspect the object’s contents. `sized()` checks a vector object’s length, `complete()` checks that a list contains no NULL values and a vector contains no NA values, `bounded()` checks that all elements of a vector fall within a given range, `unduplicated()` checks that the object contains no repeated elements, and `within()`, `contains()`, `same_as()`, `setequal_to()`, and `disjoint_to()` check set-membership between an object and a reference set.

Traits are designed to be composed using the native pipe operator `|>`. A new trait adds a requirement to the left-hand side type, allowing new types to be defined incrementally and existing types to be refined with additional restrictions.

```
t_probability <- t_any |> # Any object in R
  bare_typed("double") |> # With a `typeof()` of "double"
  bounded(0, 1, "[")      # Bounded between 0 and 1 inclusive
```

In addition to the `t_any` type, the package provides built-in types for common objects, including `t_bool` (a scalar non-NA logical), `t_string` (a scalar non-NA character), `t_vec` (a

`vctrs` (Wickham et al., 2026) style vector), `t_int` (a bare integer vector), `t_dbl` (a bare double vector), `t_chr` (a bare character vector), `t_lgl` (a bare logical vector), `t_list` (a list), and `t_fun` (a function). All exported types are prefixed with `t_` and, where possible, use suffixes, such as `chr` for character, established by popular packages including `purrr` (Wickham and Henry, 2025) and `rlang` (Henry and Wickham, 2026) to make built-in types recognizable by name.

Type construction is designed to be self-descriptive and simple to use. For example, `t_bool` is defined as `t_lgl |> sized(1L) |> complete()`, expressing that a boolean value must be a bare logical vector (`t_lgl`) of size 1 (`sized(1L)`) containing no missing values (`complete()`).

### 3.2 Checking Type Membership

Three functions inspect the type membership of an object:

- `obj_is_type(obj, type)` returns a `TRUE` if `obj` is a member of `type` and `FALSE` otherwise.
- `obj_assert_type(obj, type)` raises an error if `obj` is not a member of `type`, using `rlang` and `cli` (Csárdi, 2025) to produce a Tidyverse style guide (Wickham, 2023) compliant error message.
- `obj_inspect_type(obj, type)` prints a per-trait diagnostic, showing which traits passed, which failed, and why.

These functions provide a consistent interface to determine the runtime type of an object in any context. `obj_inspect_type()` is the primary tool for interactive type introspection.

```
obj_inspect_type(NA_character_, t_string)
#> Object `NA_character_` does not have the expected type.
#> v `NA_character_` is a bare <character>.
#> v `NA_character_` is size 1.
#> i `NA_character_` must not contain missing elements.
#> x `NA_character_` is NA at location `1`.
```

Each line corresponds to one trait in the trait list of `t_string`, evaluated in order. Passing traits are shown with a check mark (rendered here as a `v`), using the description generated by `trait_describe()`, and the failing trait produces an informational message (what was required) and an error message (what caused the failure) generated by `trait_diagnose()`. When `obj_assert_type()` raises an error, the failing trait is reported in the same format.

A `type_union()` expresses a disjunction, with an object satisfying the union if it satisfies at least one of the member types. Failed union checks report the unmet requirements of each member separately:

```
t_rownames <- type_union(t_int, t_chr) |> complete()
obj_inspect_type(NA_integer_, t_rownames)
#> Object `NA_integer_` does not have the expected type.
#> * Type option 1 of 2:
#> v `NA_integer_` is a bare <integer>.
#> i `NA_integer_` must not contain missing elements.
#> x `NA_integer_` is NA at location `1`.
#> * Type option 2 of 2:
#> x `NA_integer_` must be a bare <character>, not a bare <integer>.
```

This diagnostic naturally surfaces the most plausible intended type of a mistyped argument or object, guiding the user towards the right correction at the cost of a more verbose message. Type unions likewise print the requirements of each of their member types.

```

print(t_rownames)
#> <type_union>
#> i Type 1 of 2:
#> * `<object>` is a bare <integer>.
#> * `<object>` contains no missing values.
#> i Type 2 of 2:
#> * `<object>` is a bare <character>.
#> * `<object>` contains no missing values.

```

### 3.3 Structural and Container Types

The traits described in Section 3.1 evaluate an object as a whole. For types that constrain parts of an object, for instance its elements, attributes, or names, **type** provides a small dialect of selectors.

The `has(type, selector, on_type)` trait adds a constraint requiring the part of an object specified by the selector to be a member of the `on_type` type. Selectors identify parts of an object using accessors:

- `on(accessor)` selects the result of calling the function `accessor` on the object (e.g. `on(names)` selects `names(obj)`).
- `on_elm(index)` selects an element by name or location, e.g. `obj[["x"]]` or `obj[[1L]]`.
- `on_attr(attr)` selects an attribute, e.g. `attr(obj, "levels")`.
- `on_data()` selects the base data of an object, after removing its class and other attributes.
- `on_each()` selects every element of a vector object, requiring each element to be a member of the specified type.

The `on_type` applied to the selected part of an object may itself have `has()` constraints, enabling types of arbitrary nested depth.

`has_relation(type, relation)` constrains how selected parts of an object relate to one another, allowing between-element type requirements. Four relations are currently provided:

- `same_sized(...)` requires all selected parts to have the same size, analogous to the `sized()` trait.
- `same_classed(...)` requires all selected parts to share the same `class()`.
- `recyclable(...)` requires all selected parts to either share the same size or be of size 1.
- `exclusive(...)` requires that exactly one selected part be non-NULL.

The `has()` and `has_relation()` selection dialect is sufficiently expressive to replicate the requirements of a valid data frame enforced by the `data.frame()` constructor (see Section 2).

```

t_dataframe <- t_any |>
  classed("data.frame") |>
  has(on_data(), t_list) |>
  has(on_each(), t_vec) |>
  has(on_attr("row.names"), type_union(t_chr, t_int) |> complete()) |>
  has_relation(same_sized(on_each(), on_attr("row.names")))

```

The definition above may be used to validate that an arbitrary object is a data frame in any context, catching cases of post-construction mutation which `is.data.frame()` would fail to detect.

```
# Create a malformed data frame after construction
```

```
obj <- data.frame(x = 1:3, y = c("a", "b", "c"))
attr(obj, "row.names") <- 1L

# Later, assert type membership
obj_assert_type(obj, t_dataframe)
#> Error in `obj_assert_type()`:
#> ! Object `obj` is mistyped.
#> i `obj[["x"]]' and `attr(obj, "row.names")` must be the same size.
#> x `obj[["x"]]' is size 3 and `attr(obj, "row.names")` is size 1.
#> i Run last_type() to get the expected type.
```

Declaring `t_dataframe` as a standalone type object makes the definition of a valid data frame explicit and inspectable. Printing the type displays each requirement in plain language, with no need to read a constructor body or a sequence of assertions to infer the type definition. Further, the type is portable, allowing it to be passed as a value, stored, and re-used across multiple typed functions or typed object declarations without duplication. Lastly, users can refine the `t_dataframe` type incrementally without redefining the base requirements:

```
# Constrain the type to further require `id` and `score` columns
t_survey <- t_dataframe |>
  has(on_elm("id"), t_int |> complete()) |>
  has(on_elm("score"), t_dbl |> bounded(0, 100))
```

For the common cases of typed lists and typed data frames, **type** provides shorthand constructors:

```
t_survey <- dataframe_type(
  id = t_int |> complete(),
  score = t_dbl |> bounded(0, 100)
)

t_pair <- list_type(x = t_dbl, y = t_dbl)

t_listof_pair <- list_of_type(t_pair)
```

`dataframe_type()` builds a type requiring a "data.frame" class, specific named columns of the given types, and consistent row counts across all columns. `list_type()` builds a type for a named list with specific element types. `list_of_type()` covers the common case of a homogeneous list where every element must satisfy the same type.

### 3.4 Typed Functions

The most common use of **type** is for annotating function arguments, using a terse annotation syntax inspired by Python's type hinting system (van Rossum et al., 2014). The `typed()` function wraps a function declaration, allowing argument and return values to be annotated with a type. Argument types are declared by replacing the default value with a type expression, and default values may be provided instead using the `%%` operator:

```
safe_log <- typed(function(x = t_num, base = t_num %% exp(1)) {
  log(x, base = base)
})
```

When `safe_log()` is called, each typed argument is checked before the function body executes. Mistyped arguments raise an error immediately at the call site, automatically enforcing a defensive programming style:

```
safe_log("ten")
#> Error in `safe_log()`:
#> ! Argument `x` is mistyped.
#> x `x` must be a numeric vector, not the string "ten".
#> i Run last_type() to get the expected type.
```

The dots (...) argument may be typed per element, with ... = t\_lgl requiring that each supplied dot is a bare logical, or treated as a single list using the t\_dots type. ... = t\_dots |> sized(2L) for example requires that exactly two dots be supplied.

The return type of a function is specified using the returns argument:

```
safe_any <- typed(
  function(...) = t_lgl, na.rm = t_bool %:% FALSE) {
    any(..., na.rm = na.rm)
  },
  returns = t_lgl |> sized(1L)
)
```

All relations accepted by has\_relation() (see Section 3.3) may additionally be used to constrain how multiple arguments relate to one another. A relation call may be placed before or after the function definition:

```
na_string <- typed(
  recyclable(x, na_to),
  function(x = t_chr, na_to = t_chr |> complete()) {
    x[is.na(x)] <- na_to
    x
  },
  returns = t_chr |> complete()
)
```

```
na_string(c("hi", NA, "bye"), na_to = "N/A")
#> [1] "hi" "N/A" "bye"
na_string(c("hi", NA), na_to = character())
#> Error in `na_string()`:
#> ! Arguments `x` and `na_to` must be recyclable.
#> x `x` (size 2) and `na_to` (size 0) are incompatible sizes.
```

When a typed function is printed, its argument and return types are displayed alongside the function body, providing documentation of type expectations at no additional documentation cost:

```
print(safe_any)
#> <typed>
#> function (... , na.rm = FALSE)
#> {
#>   any(... , na.rm = na.rm)
#> }
#> Arguments:
#> * Each element of `...` is a bare <logical>.
#> * `na.rm` is a bare <logical>.
#> * `na.rm` is size 1.
#> * `na.rm` contains no missing values.
#> Returns:
#> * `<result>` is a bare <logical>.
#> * `<result>` is size 1.
```

The expected type from the most recent failed type check is always accessible via `last_type()`, which is useful when the error message alone does not provide the call at which the error occurred, as is the case when using `lapply()` for example:

```
# The error does not mention safe_any()
lapply(list(c(TRUE, FALSE), NA), safe_any, na.rm = "no")
#> Error in `FUN()` :
#> ! Argument `na.rm` is mistyped.
#> x `na.rm` must be a bare <logical>, not a bare <character>.
#> i Run last_type() to get the expected type.

# Retrieve the full expected type specification
last_type()
#> <type>
#> * `<object>` is a bare <logical>.
#> * `<object>` is size 1.
#> * `<object>` contains no missing values.
```

`untyped()` may be used to remove the argument and return type assertions of any typed function, while maintaining its original behaviour:

```
fast_any <- untyped(safe_any)
print(fast_any)
#> function (... , na.rm = FALSE)
#> {
#>   any(... , na.rm = na.rm)
#> }
```

This supports a gradual adoption workflow, allowing type annotations to be added during development for clear error messages and documentation, then selectively removed when input data is pre-validated or in performance-sensitive paths.

### 3.5 Typed Variables and Constants

The `%%` operator is used to declare a typed variable. The right-hand side must be a declaration of the form `name(value)`, where `name` becomes a variable name and is assigned value as its initial value. After declaration, every subsequent assignment to the typed variable is type-checked:

```
t_int %% count(0L)
count <- "A"
#> Error:
#> ! Attempted to assign a mistyped value to `count`.
#> x `<value>` must be a bare <integer>, not a bare <character>.
#> i Run last_type() to get the expected type.
```

Wrapping the left-hand side type in `const()` prevents re-assignment entirely, producing a constant:

```
const(t_string) %% version("1.0.0")
version <- "1.1.0"
#> Error:
#> ! Can't assign to the constant `version`.
#> i Run last_type() to get the expected type.
```

### 3.6 Summary

Taken together, the type system allows users to define custom data types using a concise, readable syntax, without the need for a formal class constructor or hand-written validation functions:

```
# A set of double values
t_dbl_set <- t_dbl |> complete() |> unduplicated()

# A labelled vector
t_labelled <- t_vec |> has(on_attr("label"), t_string)
```

From a single definition, the user gains a self-documenting type description, interactive type membership tests via `obj_is_type()` and `obj_inspect_type()`, reusable enforcement at any function boundary via `typed()`, and informative error messages. As the requirements of a project grow, existing types can be extended and composed rather than redefined. A type that permits NULL in addition to a double set, or a homogeneous list of double sets, is expressed by building on the original definition rather than duplicating it:

```
t_maybe_dbl_set <- type_union(t_null, t_dbl_set)
t_listof_dbl_set <- list_of_type(t_dbl_set)
```

Any subsequent changes to `t_dbl_set`, adding or removing an additional trait for example, propagate to every type built from it. As types are defined explicitly as code, they may be unit tested, updated over time, exported for use across projects, and version controlled.

## 4 Related Work

**typed** (Fabri, 2021) is the closest existing tool for adding argument and object type annotations to R. The package defines types using function factories, rather than as standalone objects, and overrides the `?` operator for type declaration:

```
# Declaring a typed character (typed)
library(typed)
Character() ? z <- "A"
z <- 1L
#> Error:
#> ! type mismatch
#> `typeof(value)`: "integer"
#> `expected`:      "character"

# Declaring a typed character (type)
t_chr %:> z("A")
z <- 1L
#> Error:
#> ! Attempted to assign a mistyped value to `z`.
#> x `<value>` must be a bare <character>, not a bare <integer>.
#> i Run last_type() to get the expected type.
```

**typed** does not support type unions, element sub-typing (e.g. via `has()`), or type composition (e.g. via `list_of_type()`). While **type** currently supports only the fixed set of trait predicates, **typed** supports custom validation predicates within typed generator functions. For example, the type assertion expression `Character(... = "Must be length 1" ~ length(.) == 1L)` adds an additional requirement that typed character vectors are length 1.

Several packages exist for adding runtime argument validation in R using assertion functions. **checkmate** for example, provides a large collection of atomic check functions (`assertNumeric()`, `assertCharacter()`, etc.) with fast C implementations, and extends the **testthat** (Wickham, 2011) unit testing framework with type testing functions. Unlike **type**, checks in **checkmate** are standalone functions meant to support the defensive programming pattern, and do not compose into reusable or extensible type objects. The **type** package is slower than **checkmate**, which uses optimized compiled code for its checks, although `untyped()` provides an escape hatch for performance-critical paths at the cost of foregoing runtime type validation.

**validate** (van der Loo and de Jonge, 2021) provides a rule-based system for expressing and evaluating data quality constraints on data frames, with a domain-specific language for between-variable and between-row restrictions. The package is complementary to **type**, as its target use case is for dataset validation in data processing pipelines rather than function argument or variable typing, and supports only checks on the data frame class. **assertr** (Fischetti, 2023) similarly targets pipeline validation of data frames, providing assertions that can be inserted into native `|>` chains.

Python’s optional type hinting system (van Rossum et al., 2014) is the closest cross-language analogue to **type**’s usage goals. Python’s type hints are optional, incrementally adoptable, and do not require rewriting existing code, the same approach supported by function annotation with `typed()` and `untyped()`. The **pydantic** (Colvin et al., 2026) library extends Python’s type hints with runtime type enforcement via validator functions, for example constraining an integer type to only positive values, similar to **type**’s trait system.

## 5 Limitations

**type** offers a limited set of traits and relations (`same_sized()`, `same_classed()`, `recyclable()`, and `exclusive()`) and does not yet support custom user-defined validation functions for either traits or relations. Given that types are composed incrementally, additional traits or relations may be added to the package without altering existing code, and all existing types will immediately support composition with new traits. Additionally, `type_union()` does not currently support member types that carry `has()` traits, meaning a union of two `dataframe_type()` objects for example is not yet expressible.

**type** checks are currently slower than other runtime type assertion packages, in particular **checkmate**, as every type membership test requires walking the full list of traits held by a type and evaluating a call to `trait_test()` for each trait. The current implementation of `typed()` inserts the equivalent of `obj_assert_type(arg, arg_type)` into the function body for each typed argument, which walks the full trait list of `arg_type` on every call of the typed function. A future implementation may reduce this overhead by inlining fast test expressions directly into the function body at definition time, for example replacing the full trait-list evaluation of `arg = t_chr |> sized(1L)` with a test `is.character(arg) && length(arg) == 1L`, avoiding the overhead of trait iteration in the case where `arg` is correctly typed. The same approach may be used for typed objects declared with `%>`.

## 6 Conclusion

**type** provides a lightweight, composable runtime type system for R. Types are defined as sets of predicate traits, which compose nominal (`class()`), structural (`typeof()`), and value-level checks to describe the restrictions a valid object must satisfy. The type system addresses two failure modes of R’s permissive nominal typing. First, **type**’s explicit type definitions can detect a structurally malformed object of a given class at any point in the object’s lifecycle, rather than only at construction, which class-based checks fail to identify. Second, **type** prevents the propagation of type errors through function calls by validating argument types immediately at the function boundary, producing error messages that identify the erroneous call, mistyped argument, the constraint it violated, and the cause of the violation.

Unlike the standalone assertion functions used in defensive programming, types in **type** are self-documenting, with both types and typed functions displaying their requirements when printed. As first-class R objects, types can be easily inspected, unit tested, composed, and exported, allowing a project's type vocabulary to grow and be improved incrementally alongside its codebase.

## References

- S. Colvin, E. Jolibois, H. Ramezani, A. Garcia Badaracco, T. Dorsey, D. Montague, S. Matveenko, M. Trylesinski, S. Runkle, D. Hewitt, A. Hall, and V. Plot. Pydantic validation, 2026. URL <https://pydantic.dev/docs/validation/latest/get-started/>. [p11]
- G. Csárdi. *cli: Helpers for Developing Command Line Interfaces*, 2025. URL <https://CRAN.R-project.org/package=cli>. R package version 3.6.5. [p5]
- A. Fabri. *typed: Support Types for Variables, Arguments, and Return Values*, 2021. URL <https://CRAN.R-project.org/package=typed>. R package version 0.0.1. [p10]
- T. Fischetti. *assertr: Assertive Programming for R Analysis Pipelines*, 2023. URL <https://CRAN.R-project.org/package=assertr>. R package version 3.0.1. [p11]
- L. Henry and H. Wickham. *rlang: Functions for Base Types and Core R and 'Tidyverse' Features*, 2026. URL <https://CRAN.R-project.org/package=rlang>. R package version 1.1.7. [p5]
- W. M. Landau. The targets r package: a dynamic make-like function-oriented pipeline toolkit for reproducibility and high-performance computing. *Journal of Open Source Software*, 6(57):2959, 2021. URL <https://doi.org/10.21105/joss.02959>. [p2]
- M. Lang. checkmate: Fast argument checks for defensive R programming. *The R Journal*, 9(1):437–445, 2017. doi: 10.32614/RJ-2017-028. [p2]
- A. Turcotte, A. Goel, F. Křikava, and J. Vitek. Designing types for r, empirically. *Proc. ACM Program. Lang.*, 4(OOPSLA), Nov. 2020. doi: 10.1145/3428249. URL <https://doi.org/10.1145/3428249>. [p1]
- M. P. J. van der Loo and E. de Jonge. Data validation infrastructure for R. *Journal of Statistical Software*, 97(10):1–31, 2021. doi: 10.18637/jss.v097.i10. [p11]
- G. van Rossum, J. Lehtosalo, and L. Langa. PEP 484 – Type Hints. Python Enhancement Proposals, 2014. URL <https://peps.python.org/pep-0484/>. [p7, 11]
- D. Vaughan, J. Hester, T. Kalinowski, W. Landau, M. Lawrence, M. Maechler, L. Tierney, and H. Wickham. *S7: An Object Oriented System Meant to Become a Successor to S3 and S4*, 2025. URL <https://CRAN.R-project.org/package=S7>. R package version 0.2.1. [p4]
- M. Vidoni. Understanding roxygen package documentation in r. *Journal of Systems and Software*, 188:111265, 2022. ISSN 0164-1212. doi: <https://doi.org/10.1016/j.jss.2022.111265>. URL <https://www.sciencedirect.com/science/article/pii/S016412122200310>. [p2]
- H. Wickham. testthat: Get started with testing. *The R Journal*, 3:5–10, 2011. URL <https://journal.r-project.org/articles/RJ-2011-002/>. [p11]
- H. Wickham. The tidyverse style guide, 2023. URL <https://style.tidyverse.org/>. [p5]
- H. Wickham and L. Henry. *purrr: Functional Programming Tools*, 2025. URL <https://CRAN.R-project.org/package=purrr>. R package version 1.2.0. [p5]
- H. Wickham, L. Henry, and D. Vaughan. *vctrs: Vector Helpers*, 2026. URL <https://CRAN.R-project.org/package=vctrs>. R package version 0.7.1. [p5]

*Ethan Sansom*  
*University of Toronto*